

Evaluating Decision Tree Models for Predictive Analytics

RIJAN POKHREL¹, (Student, IOE), and SABIN ACHARYA², (Student, IOE)

¹Department of Electronics and Computer Engineering, Thapathali Campus, Thapathali, Nepal (e-mail: rijan.077bei034@tcioe.edu.np)

²Department of Electronics and Computer Engineering, Thapathali Campus, Thapathali, Nepal (e-mail: sabin.077bei035@tcioe.edu.np)

This lab was performed under the supervision of Er. Dinesh Baniya Kshetri in Department of Electronics and Computer Engineering, Thapathali Campus, IOE, T.U.

• **ABSTRACT** In this lab report, the Decision Tree Classifier is trained and tested on two different datasets i.e. Weather Dataset and Titanic Dataset. The classifier is trained by changing different hyperparameters like the measures of split (entropy, gini and logloss), and depth of the tree and then analyzed to reach certain conclusions. This report uses different preprocessing techniques to tackle the ununiformness in the data in the dataset. Also cross validation techniques are also used to find the best set of parameters to get the best accuracy from the model.

• **INDEX TERMS** Binary Cross Entropy, Cross Validation, Early Stopping, Entropy, Gain Ratio, Gini, Information Gain, Pruning, Splitting Criterion

I. INTRODUCTION

IN data science, classification is the task of assigning predefined categories or labels to input data based on its characteristics or features. The classification model is created using the training data that contains the attributes and class for each training instances. The method of learning used to train a classification model from given data is known as a learning algorithm. The process of using a learning algorithm to build a classification model from training data is known as **induction** and the process of applying a classification model on the test data to predict their class is known as **deduction**. The process of classification thus undergoes the following two steps:

- 1) Learning from the training data to build a model.
- 2) Applying the same model to classify the test data.

Classification are essentials in various fields like medical diagnosis, spam detection, credit scoring, customer churn etc.

A. CLASSIFICATION ALGORITHM FUNDAMENTALS

Classification algorithms are the milestone for supervised machine learning. It is especially designed to predict outcomes for new data based on the patterns it learned from the trained data. Classification Algorithms analyze the input features to find the relationships which differentiate between different classes or categories. The classification process creates a model that can map the input data to the respective class labels by identifying optimal decision boundaries in the features. Many classification algorithms give not only the predictions but also the probabilities estimates offering enough information for the decision making process.

The scope of the classifications algorithms is diverse i.e. equally applicable for simple linear models to complex non-linear models. The most common methods for classifications are: Logistic regression, decision trees, Support Vector Machines(SVM), Artificial Neural Network(ANN) etc. The choice of these algorithms depends on various factors: DataSet Characteristics, desired interpretability and respective problem requirements.

Classification algorithms are widely used in various domains such as medical domain, finance domain, image recognition etc. It is mainly used as an automated decision making tool and pattern recognition in many real world problems.

B. TYPES OF CLASSIFICATION

Generally, classification can be categorized into binary(two features) or multi-class(more than two classes). Also, a special type of classification called hierarchical classification is also used in some cases where class have heirarchical structure. The categories are further explained as follows:

1) Binary Classification

It is the simplest form of the classification where the class has only two outcomes. For example: Sentiment Analysis (Negative or Positive), Spam Detection (spam or not spam), Disease Prediction (disease present or absent) etc. Mainly, Logistic regression and SVM are used for the binary class classification problems.

2) Multi-class Classification

It is the classification where the output class has more than two classes. For example: Classifying the genre of the movie (it can be comedy, thriller, action and many more), categorizing news articles by topics, identifying species of plants, etc. Decision trees, Random forest, and Neural Network support the multi class classification.

3) Hierarchical Classification

Some of the classes have a hierarchical structure. For example: for the classification of the animal it may involve top-level categories i.e. Vertebrates, Invertebrates and then sub-categories within each like Mammal, Reptile, Amphibian inside the Vertebrates and like Arthropoda, Mollusca, etc inside the Invertebrates further. This type of classification requires the special approach that respects the class hierarchy.

C. DECISION TREE

A decision tree is a tree that takes decisions to classify a data instance to a certain class. It is built on the idea that a complex problem can be solved by dividing it into several parts and solving them.

For a given input data, a decision tree estimates an unknown property of the data y by asking a series of questions about its known properties x . A decision tree can be used for both classification and regression tasks. Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand. Decision Trees takes its structural inspiration from a tree. It commences from a singular node having no parent node known as **root node** from which different other branches emerge. The branch with single parent and multiple children is known as **internal node**. Similarly, the branch with no children node are known as **leaf nodes**. The leaf nodes ends the tree traversal by providing a specific class label. The internal nodes and root node contain the attribute test conditions for a single attribute. This format allows the mapping of how decisions and actions will lead to different outcomes in the future.

D. TYPES OF DECISION TREE

Based on the type of outcome variables used, there are two main types of decision trees: categorical and continuous.

1) Categorical Variable Decision Tree

In a categorical variable decision tree, the answer neatly fits into one category or another. For example: Was the coin toss heads or tails? Is the animal a reptile or mammal? In this type of decision tree, data is placed into a single category based on the decisions at the nodes throughout the tree.

2) Continuous Variable Decision Tree

A continuous variable decision tree is one where there is not a simple yes or no answer. It is also known as a regression tree because the decision or outcome variable depends on other decisions farther up the tree or the type of choice involved in the decision.

The benefit of a continuous variable decision tree is that the outcome can be predicted based on multiple variables rather than on a single variable as in a categorical variable decision tree. Continuous variable decision trees are used to create predictions. The system can be used for both linear and non-linear relationships if the correct algorithm is selected.

II. LITERATURE REVIEW

The paper [1] by Zhao and Nie addresses some important theoretical questions about decision trees: bias in the splitting criterion, effects of noise variables, and computational complexity. The authors have shown that the Gini index is a better splitting criterion than entropy information gain since it is less biased in case of missingness and when the number of categories in a variable is large. It is shown that noise variables with more missing values are more likely to be chosen as splitting variables, and informative variables with more missing values are less likely to be selected. In this work, the computational complexity of the CART(Classification and Regression Tree) algorithm is addressed, explaining that it linearly goes up once noise variables are added. Such findings offer some useful insights into the behavior and limitations of decision tree algorithms, especially CART, along with guidelines for preprocessing steps that can be applied in practice. Realizing that their analysis relies on simplified assumptions, the authors further suggest research in this direction to take into account more complex cases. In general, this paper has contributed significantly to the theoretical understanding of decision tree algorithms and ways for their effective implementation in various fields.

The paper [2] describes a generalized way to unify classical split criteria in decision trees with Tsallis entropy. The authors give a Tsallis Entropy Criterion algorithm with an adjustable parameter q , from which traditional split criteria like Shannon entropy, Gain Ratio, or Gini index can be deduced. In this paper, the key contribution is to prove the relationships between Tsallis entropy defined for various q values and classical criteria. Experimental results on the UCI datasets demonstrate that the TEC algorithm is statistically significantly more accurate, on average, by 4% for appropriate values of q , keeping the tree complexity at a lower level. Besides, the authors notice a possible trend in the optimal q with respect to dataset properties, like the number of classes. The work thus goes on to provide a unifying framework for decision tree split criteria and further points out one flexible avenue

for improving decision tree performance using tunable parameter q in Tsallis entropy.

The paper [3] addresses the preprocessing phase of data mining. It addresses discretization of numerical attributes and grouping of nominal ones. The authors define "offline" algorithms for these tasks, executed prior to the actual data mining step. Contrary to "online" discretization methods applied in many machine-learning algorithms, these off-line approaches process data independently of the learning algorithm. The discretization algorithm creates the intervals where there are considerable differences in the distribution of classes with respect to the overall class distribution, while the grouping algorithm makes a fixed number of groups corresponding to the number of classes. Both algorithms are class-sensitive; thus, they are very appropriate for classification and prediction tasks. The testing of their methods against a number of UCI datasets and the KDD Sisyphus dataset also showed results quite comparable to other machine-learning algorithms, including CN4 and C4.5. They concluded that their preprocessing techniques can reduce considerably the number of objects and attributes in the datasets without much effect on classification accuracy. Also, these methods are potentially very good at spotting irrelevant attributes and delivering interpretable knowledge regarding the data. On the whole, the developed machine learning algorithm suggests that it is an off-line preprocessing approach of real value for many algorithms—not just the KEX system for which it was developed.

The book [4] proposes different types of discretization techniques and the impurity checking techniques among which the supervised discretization technique using entropy was used for this lab report. Similarly, it made more clear about the process to find the best splitting to make the continuous data to categorical form.

III. METHODOLOGY

The System Block Diagram of the Methodology used for classification problem is shown in figure (1). It consists of 3 major blocks: Data Preprocessing Pipeline which makes the raw dataset feasible for model training, Model Training Block which se-

lects different training hyperparameters and trains the model on train data and lastly the Model Evaluation Block which evaluates the trained model's performance by predicting test data's classes and using different evaluation metrics.

A. DATASET OBSERVATION

This fundamental step is used to gather the basic insights on the dataset quality analysing if the provided features give the necessary insight to predict the class of the data instance. Having more random features that have no role in classifying the data instances such the values unique to each data instances like Citizenship Number, or the features that may be common but has no impact on classifying data like names, etc can make the model confuse by trying to find the correlation between the output class and those features. These features just act as the noise to model that when removed increases the model's performance. This can be done by knowing what the dataset is about and what can be the unnecessary columns using common understanding of the problem domain.

B. REMOVAL OF UNNECESSARY COLUMNS

This step as its name suggests is all about dropping the columns/features that is found to have no relation with categorization process. It reduces the dimension of the dataset saving the model from the curse of dimensionality and also from the noise.

C. VISUALIZATION AND ANALYSIS OF DATASET

This step uses some statistical and diagrammatic tools to get indepth insight on the dataset features. The dataset is examined if it has any specific outliers that can greatly impact the mean. The sparsity of the dataset is checked by generating the count of null values in each of the column. The datatype of the features is also examined which allows to determine if there is some requirement of the discretization or not. These insights can be used to further pre-process the dataset.

D. HANDLING OF NULL VALUES

The decision tree classifier cannot accurately make the decision tree in presence of null values in features set. It just ignores the null values and make the decision tree which can infer to a different

prediction strategy by the model. So, it is required to fill the null values with appropriate values. There are several strategies that can be applied to handle the null values. Some of them are:

a: Removing Data instance with null values

This is done when there are only a very few instances with null values in comparison to the total instances.

b: Imputing

It refers to placing the value of the central values like mean, median, mode of the features in place of null values. The mean is placed if the dataset consists of less number of outliers, the median if the feature consists of large number of outliers and the mode is place if the feature is categorical in nature.

c: Using K-Nearest Neighbours

This method calculates the 'k' closest neighbours of the null value and places the class in which most of the neighbours belong. It is also only applicable for categorical data.

d: Manual Imputation using Domain Knowledge

If a person has expertise in the problem domain, then using that expertise, the data can be imputed manually by the person. This imputation performed by non-experts can lead to degradation of dataset quality.

E. DISCRETIZATION OF CONTINUOUS VARIABLES

It is a data preprocessing process to transform continuous numerical variables into discrete categorical features. It divides the range of continuous data into a finite number of intervals or bins, each representing a category. It is especially used in certain classification algorithms such as decision tree algorithms or Naive Bayes Classifiers which require the data to be in the form of categorical form. It is also helpful to reduce the effects of small fluctuations in continuous variables. There are various methods for discretization, broadly categorized into unsupervised and supervised approaches. The choice of discretization can impact the performance of the corresponding ML model. Discretization should be applied wisely, as it can also lead to loss

of the information when numerical relationships are important for the classification process.

1) Binarization

It is a data preprocessing technique which transforms numerical or categorical data into binary (0 or 1). In this process, typically binary indicators are created for categorical variables and a threshold value for numerical data. For continuous data values beyond the threshold is assigned 0 while above threshold are assigned 1. The value for the threshold is sensitive and can highly impact the performance of the model. It is determined using various methods such as statistical analysis of the data distribution, domain knowledge of the dataset or optimization based on model performance.

It is also important to take in consideration the potential drawback of binarization. It loses the information in the case where the magnitude of the value have significant impact. Binarization can reduce the impact of the outliers, simplify complex data and make certain algorithms more effective especially the algorithms that works with binary inputs like logistic regression or some neural networks architectures.

2) Unsupervised Discretization

It transforms the continuous variables into discrete categories without the information of class variables. It is simple to implement but doesn't create effective categorization due to lack of class information. The most common method are:

a: Equal Width/Interval Binning

This method divides the range of continuous variables into the given number of bins, of equal width or range. It is simple to implement. But it is sensitive to the outliers and does not perform well with the skewed distributions. It is very effective when the data is uniformly distributed in its range. But it may create bins with few data points where the data is concentrated in certain regions of the variables range.

b: Equal Frequency/depth Binning

It creates bins having approximately the same number of data points. This method sorts the data and divides it into N groups each containing approxi-

mately the same number of data points. It is least sensitive to the outliers and handles skewed data efficiently. However, it may group together the values that are far apart in data distribution disturbing important patterns and threshold effects in the data.

c: K-Means Discretization

It uses the K-means clustering algorithm for creating the bins. It treats the continuous data as one dimensional data and applies the K-means algorithm for identifying the clusters. The boundaries between the clusters act as a discretization cut points. It is typically useful when the data has the natural groups or clusters. However, it is sensitive to the initial selection of cluster centers and produces different results every time it runs. Also the selection of bins in advance may require domain knowledge or experiment for determining the optimal values.

3) Supervised Discretization

It transforms the continuous variables into the discrete ones by taking in account the relationship between the input and class variables. This method aims to create bins that are most informative for prediction tasks. This approach aims to maximize the information gain (minimize entropy) with respect to the class variable. The process typically starts with many small intervals and iteratively merges adjacent intervals. Merging decisions are based on which combination results in the highest information gain or the lowest entropy in relation to the target variable. However, it can be computationally intensive for large datasets and may require careful tuning to avoid overfitting.

F. HANDLING CATEGORICAL DATA

1) One Hot Encoding

It is a technique of converting the categorical data to the numerical data. It creates a new column for each unique category in original data. Each of these new columns represents the presence or absence of the specific category. It is particularly useful in linear models, neural networks and the algorithms that needs the relationships between the numerical data. However, it increases the dimensions of the data especially for categorical variables with various unique values. This lead to the curse of dimensionality and increases the computational complexity.

Also for columns with large number of categories, it may lead to the sparse matrix.

2) Label Encoding

Label Encoding is another technique for converting the categorical data into the numerical data. In this method, a unique integer value is assigned for each unique category within a variable. Label encoding preserves the ordinal relationship between the categories. Label encoding is preferred for the tree-based algorithms like random forest and decision trees that handles the numerical data without assuming the relationship between the values. It is also fruitful to use when there are various categories in the columns as it does not increase the dimensions. It is not preferred to use the label encoding in the algorithms that assumes the linear relationships such as linear regressions, neural networks because they may interpret the encoded values as having mathematical relationships among them.

G. SELECTION OF SPLITTING CRITERION

The splitting criterion in decision tree classifier measures the quality of split. Selection of different splitting criterion in a decision tree classifier can significantly impact the performance, interpretability and behaviour of the resulting model.

1) Gini

Gini gives the probability of misclassifying a data point. It is a measure of impurity in the data. A higher gini value signifies higher impurity and vice-versa. Impurity, in general, depends upon the distribution of data. Uniformly distributed data have high impurity. Non-uniform data have lower impurity. For pure data, impurity is 0 and so is the value of Gini. The value of Gini ranges from 0 to 0.5 where 0.5 denotes maximum impurity. Gini can be computed using the formula of equation-(1).

Gini is computationally less intensive and also often result in shallower trees which are less prone to overfitting than the entropy. But it may not always create the most balanced splits, potentially impacting the model performance.

2) Entropy

Entropy is another metric to quantify impurity. It is a measure of disorder or randomness in data. It can be computed using the formula of equation-(2). Its value ranges from 0 to 1.

Entropy can lead to more balanced splits which can be beneficial for certain datasets and can also handle diverse distributions of data. But entropy calculation is computationally intensive than the Gini Impurity. Gini is more preferred to be used in training than entropy when the computational availability is low and the dataset is large. Also the entropy can lead to deeper trees and lead to overfitting.

3) Classification Error

Classification error is another metric used to evaluate the impurity of a node in a decision tree. It measures the proportion of instances in a node that do not belong to the majority class. The classification error can be computed using the formula of equation-(3)

This metric gives an indication of how mixed the classes are within a node, with lower values indicating a more homogeneous node.

Classification error is less sensitive than the other impurity measures like Gini impurity and entropy. It is because it considers only the majority class, that is, not taking into consideration the distribution of the minor classes in the node. Thus, compared with Gini impurity or entropy, which gives a more detailed view on class distribution, it may miss some subtle differences in node purity. Hence, classification error is less preferred in the calculation of the impurity metrics.

4) Log Loss

Log Loss or Cross Entropy is another criterion that can be used to measure the quality of split. It measures classifier's quality by penalizing for misclassifications. It compares the predicted probability distributions against the class outcomes. A smaller log loss value is preferred, with 0 perfect models. The log loss can be calculated using the formula of equation-(4).

Log loss is especially useful in decision trees where the tree is designed to return class prob-

abilities rather than simple class labels. It gives a much finer sense of performance of the model compared to accuracy metrics since it takes into account the confidence associated with each prediction. Log loss is a very strict measure of model performance because it strongly penalizes those confident, wrong predictions. However, as with any other measure, log loss would also be included alongside other evaluation methods so that a model's performance is thoroughly realized.

H. HANDLING OVERFITTING PROBLEM

In the decision tree, nodes are splitted until all of them are pure. This results in a very deep tree causing overfitting. Unless some early stopping or pruning techniques are applied, decision trees are highly prone to overfitting. Splitting increases the complexity of the tree.

The figure-(2) above shows a tree with decreasing thickness of branch at each depth. Here the thickness represents the number of samples(subset) considered while making the split in a decision tree. In the decision tree, all training samples are considered while making a split at the root node. It is evident that the decision based on a large amount of data is robust. Once a split is made at the root node, samples are divided into the child nodes. Each child node receives a subset of samples, and the decision(split) made based on these subsets is not robust as made based on whole training samples. While going deeper, the size of samples at each node decreases. The decision(split) based on a small subset is specific to the local pattern present in those subsets. So the decisions, at the deeper part of the tree, based on the local patterns in a subset of data, cause overfitting in the decision tree.

Also the overfitting causes problem when the data with features combination not present in the original training dataset occurs in the test data. When the model is overfitted, it cannot classify the data in the apart from the feature set combination available in the dataset causing problem in prediction. It occurs because the tree begins to captures the noise, outliers and specific anomaly of the training data rather than the patterns in the data. Some methods to handle this overfitting problem are as follows:

1) Early Termination

To avoid overfitting certain early stopping criteria can be specified before building a tree. Once early stopping criteria is specified, checking of these criteria at each node while splitting must be done. If any of the early stopping criteria is met, splitting the data samples at that particular node is stopped, thus effectively controlling the tree's depth. Some of the early stopping criteria are:

a: Depth Limitation

In this method maximum depth for the tree is set initially, thus it prevents the tree from growing beyond a specified number of levels. By limiting the depth, the tree's ability to create an excessive number of splits is restricted which can control the overfitting on the training data. Depth limitation forces the tree to focus on the most significant and generalized patterns in the data. The optimal depth of the tree depends on the specific dataset and the domain knowledge. It is usually determined through the cross validation where many depth limits are tested to find the best balance. However, it is important to note that specifying the depth too low leads to underfitting where the model could not capture the underlying patterns in the data.

The complexity of a decision tree increases with the increase in depth. The complexity of a decision tree can also be expressed in terms of the number of nodes as shown in equation-(5). Thus the complexity and overfitting, both can be reduced by depth limitation.

b: Impurity Threshold

It is also a parameter that controls the growth of the tree and prevents overfitting. It sets a maximum level of purity (minimum level of impurity) for a node to have for further splitting. The impurity of the node is measured using the metrics like Gini, Entropy, Information Gain etc. The impurity threshold acts as a stopping criterion i.e. if a potential split would result in child nodes with impurity below threshold, the split is avoided, and the current node is made a leaf node.

Setting an appropriate threshold is crucial to create a balance between model complexity and accuracy. Higher threshold results in a simple tree

with less splits that reduces overfitting but risked it for underfitting. Conversely, lower threshold results for more splits and complex trees that capture peculiar information but not the patterns that may lead to overfitting.

c: Minimum Samples per node

It sets a lower limit on the number of data points to be present in a node for further splitting. It applies to both internal nodes (where splits occur) and leaf nodes (terminal nodes that provide predictions). It restricts the tree from creating the branches having very few samples. It also ensures that the leaf nodes have enough samples for making reliable predictions. It also helps in creating a more generalizable model by focusing on patterns supported by a significant portion of the data.

Setting this number higher results in underfitting as the tree might not capture important patterns in smaller subsets of data. Conversely, setting it too low results to overfitting allowing tree to create small specific branches. The best value is determined through cross validation that creates the perfect balance between model complexity and accuracy. Also, it depends on the type of dataset and the knowledge on problem domain.

2) Pruning

Early Stopping is not data driven as it is applied early before training the tree. If early stopping is applied tightly (for instance, setting max-depth to very low, say 1), it generates small and under-fitted trees and conversely if applied lightly (for instance, setting max-depth to very large value) creates large decision trees which are overfitted. Thus there is not any early information to decide the suitable value for early stopping criteria.

Thus, there is a need of a robust regularization technique which is particular used for decision tree. This technique is pruning. Pruning involves growing a decision tree to its entirety and then recursively cutting/removing the nodes or sub-tree in a bottom-up approach. If removing the node/subtree improves the tree's performance on a validation dataset, then change is kept, else reverted. Experimentally, it is seen that employing pruning methods can improve the performance of the decision tree

in the test dataset. There are various pruning techniques in the decision tree. Some of them are:

- 1) Cost-Complexity Pruning
- 2) Reduced Error Pruning
- 3) Minimum Error Pruning
- 4) Error-based Pruning (EBP)

Among these pruning techniques, the most popular one is cost-complexity Pruning. The basic working procedure of pruning is to test the performance of each subtree (sub-tree can be obtained by removing one or more nodes from the original tree) on the validation set, and selecting the one that yields the best performance. However, Cost-Complexity pruning also considers the complexity of a tree apart from performance on the validation set.

Cost-complexity pruning can be used for both classification and regression tasks. Cost-complexity pruning considers both the error (cost) and the complexity (size) of the decision tree while pruning. The complexity of a tree can be expressed in different ways, like the number of nodes or depth or number of leaf nodes. It simply measures how much error is reduced by the presence of a particular branch of a particular size. If the presence of a branch doesn't reduce the error, then prune it.

I. CREATING TRAIN-TEST SPLIT

The whole dataset is to be split into two parts i.e. Train Dataset which will be used to train the model/tree and the Test Dataset which will be used to test the tree. It is important to maintain a balance between the size of train and test data. Less training and more test data can lead to improper model training whereas more training and very less test data can lead to overfitting of model and improper calculation of model's accuracy. Generally, in practice, the test dataset is taken 20% of the whole dataset.

J. MODEL TRAINING/TREE CONSTRUCTION

It is a recursive partition process that builds tree from top to bottom. It starts with the entire dataset at the root node and then splits the data into smaller subsets at each internal node. This process starts by finding the best column to split at the root node. The best suitable column to split is selected on the basis of metrics like Information Gain, Gini Impurity,

Log Loss depending on whether it is a classification or regression tree. It evaluates all the columns and finds a possible split that reduces impurity in the target variables. After the split, the data is further divided into the child nodes. The process is applied recursively for each child node and continues until the stopping criteria such as max tree depth, achieving pure nodes, few samples in node etc is achieved. This greedy approach of construction creates a model that best fits the training data. The model is trained on the test dataset by changing the different hyperparameters to get different insights on the dataset and training process.

K. CLASS PREDICTION USING TEST DATA

The test data separated from train-test split is then used to predict the model's accuracy and performance. The test data is completely unknown to the model so can effectively test the model's accuracy even for other data in real world.

L. EVALUATION METRICS FOR CLASSIFICATION

Important classification evaluation metrics are explained in detail below:

1) Accuracy

Accuracy is the simplest of all metrics. It is the measure of the proportion of correct predictions—that is, true positives and true negatives regarding total cases inspected. This, however, can become misleading when dealing with class-imbalance issues in which one class dominates over others. Accuracy is calculated using formula shown in equation-(6).

2) Precision

Precision measures the proportion of true positives against all positive predictions. It basically gives us "Out of all the instances, how many did the model label as positive that actually were positive?" Precision is especially important when false positives are too expensive. Precision is calculated using the formula of equation-(7).

3) Recall

Recall, also referred to as sensitivity or true positive rate, answers how many actual positive instances were correctly classified. It basically gives us "Of

all actual positive instances, how many did the model correctly pick up?" It is calculated using the formula of equation-(8).

4) F1-scores

The F1-score is the harmonic average of precision and recall into one score, balancing both metrics. It is useful in cases of class imbalance in data and to find a balance between precision and recall. It is calculated using the formula of equation-(9).

5) Confusion Matrix

A confusion matrix is a combination of metrics describing the model performance: true positives, true negatives, false positives, and false negatives. It gives an overview of how well the model is performing and is, in most cases, the foundation from which many other metrics can be computed.

M. EVALUATION BY TREE DIAGRAM

The scikit learn library also allows the generation of the tree model with nodes having the impurity threshold, feature threshold values, sample size, etc, which can be used to analyze the model and evaluate its performance.

IV. RESULTS

The Weather dataset and the Titanic dataset were used to analyze decision trees. Performance evaluation was conducted across numerous metrics for these decision tree models. Furthermore, hyperparameter tuning, cross validation was carried out in order to improve the models' effectiveness in prediction and efficiency. The obtained results on both datasets are described individually as follows:

A. WEATHER DATASET

The weather dataset was given by our professor for the construction of the decision tree. It consists of 4 meteorological parameters (Outlook, Temperature, Humidity, Windy) of 13 different days to predict if the game was played or not on that day (described by class column: Play). Among 4 features, Outlook and Windy are ordinal attributes and the Temperature and Humidity are nominal attributes.

On observing the dataset, it was found that the dataset was itself clean. It didn't have any missing values. So, it didn't require any data cleaning

process. But one hot encoding was performed to convert the categorical data from these features into the numerical data. After all the data preprocessing steps the number of feature columns increased to 11.

After the preprocessing step, the data was separated into the X and Y and it was splitted into train and test datasets in the ratio of 67:33. Then, the Decision Tree Classifier algorithm with entropy as splitting criteria was applied on the training dataset to train the model. Then, the test dataset was used to predict the output. The heatmap thus obtained from this prediction is shown in figure (3). The accuracy of the model was found to be 80% with a detailed classification report as shown in table-(1). The Tree diagram thus obtained is shown in figure-(4). Every node in the tree gave information like entropy, threshold values, number of samples in the node and the values in the child node after splitting.

Similarly, the tree diagram obtained from using the splitting criterion as Gini and Log-loss with no depth provided are shown in figures-(5) and (6) respectively.

The probability report of the tree obtained by using entropy as splitting criterion and the max depths 1,2,3 and 4 are shown in tables-(3),(4), (5), (6)

Then, the splitting criteria was set to entropy, gini and log-loss and a different max depth from 1 to 10 was fitted as a hyperparameter. The corresponding accuracy reading at the different max-depth is shown in the table-(2).

Cross validation and hyper parameter tuning was performed on the same tree to find the best parameters for best accuracy. Grid search CV was done to find the best hyperparameter with CV=5. And the best combination was found to be 'criterion': 'gini', 'max_depth': 3, 'max_features': 'sqrt', 'min_samples_split': 5, 'splitter': 'best' which gave the accuracy of 80%. The tree obtained using this combination of parameters is shown in figure-(7).

B. TITANIC DATASET

The Titanic Dataset is a dataset taken from Github account 'datasciencedojo' which is also available in Kaggle Machine Machine Learning Repository

(but in a cleaned form). The dataset from the github originally provides 11 features (passenger ID, Passenger Class, Name, Gender/Sex, Age, No. of Siblings and Spouses (SibSp), No. of Parents and Children(Parch), Ticket number, Fare, Cabin and Port of Embarkation) of 891 passengers that traveled in the Titanic Ship on that disastrous day and labels them on basis of their survival (1 for Survived and 0 for Not Survived).

On analyzing the dataset, it was found that the dataset was not clean and it required some preprocessing steps. First the unnecessary columns were removed. The features like Passenger ID and Ticket Number which has no relation with the survival (obvious) were dropped. Similarly, the Cabin Features was found to be highly sparse with null values in 687 rows in a total of 891 rows was also dropped due to lack of enough information. Also the gender of the passengers and their spouse count was already known, so the feature 'Name' also served no specific purpose and was dropped. This reduced the feature set of the dataset to 7.

Also, the features like Age and Embarked Station contained null values as seen from table-(7). For Age, the mean of the age of the passengers that survived was taken and filled to the passengers that survived but no age information and the mean of age of passengers that did not survived was taken and filled to the passengers that survived but had no age information. Similar approach was used for the Embarked Station feature but the mode of data was used to fill the null values because Embarked Station contained categorical values. The null values were then completely removed as seen from the table-(8).

Also after plotting the boxplot of the age and the fare column as shown in figures-(8) and (9) respectively, outliers were seen in significant numbers. To reduce the impact of the outliers, the data in the age and fare column were compressed using the log function. After compressing the features, domination was significantly reduced as shown in figures-(10) and (11) respectively.

Among those 7 features, only 3 of the features (Pclass, Age and Embarked) were categorical while rest 4 features were continuous in nature. The vi-

sual inspection of continuous data was performed using the scatter plot as shown in figures-(12), (13), (14) and (15) to find the best split position to categorize the data. But the plot didn't show any specific point of separation for best split point visually. So, the continuous data was converted to categorical using supervised binarization by calculating the best split position that created maximum information gain. The function to find the best split was self coded that could find the best split position by splitting the features into 2 parts best classifying the data as shown in code snippet-(1). The point of maximum entropy and the corresponding entropy for the features Age, Fare, SibSp and Parch was found to be 3.44, 2.44, 4 and 1 with the corresponding entropy as 0.93, 0.89, 0.95 and 0.945 respectively where the entropy of the parent was 0.9607. The data values below the split position was classified as 0 and above that split position was classified as 1. Now, One hot encoding was performed on all the categorical features which lead the total number of features to be 22.

After all these preprocessing steps, the data was separated into the X and Y and it was splitted into train and test datasets in the ratio of 80:20. Then, the Decision Tree Classifier algorithm with entropy as splitting criteria was applied on the training dataset to train the model. Then, the test dataset was used to predict the output. The heatmap thus obtained from this prediction are shown in figure-(16). The accuracy of the model was found to be 84.35%. The detailed classification report is as shown in the table-(9). The Tree diagram thus obtained is shown in figure-(17).

Similarly, the tree diagram obtained from using the splitting criterion as Gini and Log-loss with no depth provided are shown in figures-(18) and (19) respectively.

Then, the splitting criteria was set to entropy, gini and log-loss and a different max depth from 1 to 10 was fitted as a hyperparameter. The corresponding accuracy reading at the different max-depth is shown in the table-(13).

Cross validation and hyper parameter tuning was performed on the same tree to find the best parameters for best accuracy. Grid search CV was done

to find the best hyperparameter with CV=5. And the best combination was found to be 'criterion': 'gini', 'max_depth': 4 which gave the accuracy of 82.30%. The tree obtained using this combination of parameters is shown in figure-(20).

V. DISCUSSION AND ANALYSIS

From the above obtained results, various information about the decision tree classifier's working and proper training method can be deduced. The discussion on the results from both datasets are discussed separately as follows:

A. WEATHER DATASET

From the tree diagrams of the figures-(4), (5) and (6), it is evident that the decision tree will tend to go to its maximum depth and overfit in absence of any stopping criterion (max-depth in this case). The full depth the model could reach in this case was 4. The model overfitted and was 100% sure on its prediction which can be seen from the predicted probability table-(6). Each column in the matrix represented the probabilities for each class. We can see that the model is 100% confident that the class for first 3 test data was 'Yes' and that for rest 2 data was 'No'. Another reason for the model to be overfitted so quickly is due to presence of very small amount of training data i.e. just 8 data points that allowed the model to learn all the possible combinations for the training set and be sure that it will predict the exact result from dataset that it was trained on. This model will perform worse if the test dataset contain a lot of datapoints with feature values combination that is not present in the dataset.

Also applying the max-depth stopping criteria to 3, 2 and 1, the model gradually became less and less unsure about its prediction as shown in predicted probability tables-(5), (4) and (3) respectively. when the max-depth was set to 1, the model was only 50% sure on half of its prediction. This clearly shows the condition of underfitting. With depth = 1, the model instantly reached its conclusion by only checking one of the features which made it unsure about its prediction. The model predicted 2 datapoints with 100% probability in this underfitted condition too because the 2 datapoints

contained that value in its first splitted feature that were purely segregated as 'No' in the training dataset based on the first feature the tree splitted on.

Also when the splitting criteria was changed, the splitting feature the model took in each depth is different which can be seen clearly from figures-(4), (5) and (6). The model calculated the impurity of the node based on the splitting criterion that it was provided and its specific value of impurity can be seen on each node in the figures. The tree also shows the specific value of the feature that the model took to split the data points into two categories. Since the one hot encoding created all the columns with only 0 and 1 values, the model splitted all the values from the middle point of these two values i.e. 0.5 in all the nodes. The tree also shows the overfitted condition as we can see that all the leaf nodes in these 3 trees had impurity equal to 0 (i.e. entropy = 0, gini = 0 and log-loss = 0) which means that the model made the pure leaf nodes, which is the condition for overfitting. Also the model kept splitting the node wehn when the sample size was just 2 to make the purer nodes. This is all due to lack of stopping criteria for the model. Although the model perfomed well with accuracy of 80% in these 3 conditions which was the highest accuracy that was achieved, the tree formed was very deep.

The table-(2) shows that the 80% accuracy could be achieved with only maxdepth of 3 when using entropy, 2 when using gini and 2 when using log-loss. Doing this would also avoid data overfitting.

Also from the cross validation performed using grid search CV using the parameters as shown in code snippet (2). The tree for with parameters with best accuracy is shown in figure-(7). It has only depth of 3 with gini as the splitting criterion. This tree is far more efficient than the tree plotted before with same accuracy. Also the tree is not overfitted as the last leaf node has gini value of 0.375 which means impure categorization.

B. TITANIC DATASET

From the titanic dataset, it was found that when the dataset has a large amount of outliers, the outliers cannot be ignored and the only thing that can be done is to minimize its effect which was done

using log compression. Also the categorization of continuous features into more than 2 categories is very computation heavy and reaches complexity of $O(n^N)$ for N categories. So the continous data were categorized into 2 categories in this dataset. Also the decision tree classifier can work on continuous data too by finding the split position as it proceeds. But this make the process computation heavy to find the best split in each node. So, the best split was found was provided to the decision tree manually. This lead to satisfactory accuracy of about 80% with entropy as splitting criterion and no max-depth.

The entropy and no. of samples of the last 15 notes in this tree is shown in table-(10). It clearly shows that the model tends to overfitting with pure nodes with entropy values 0 and sample size 1. The table also shows that the model doesn't stop splitting even when the sample size is 2. This again verifies the point that the decision tree tends to overfit when no stopping criteria is provided. The last 15 values when using gini and logloss in place of entropy in above model are shown in tables-(11) and (12) respectively which show the same information.

The table-13 shows that the accuracy of the model is less both when the depth is too low and when the depth is too high. This shows the effect of underfitting and overfitting. Similarly, the accuracy obtained from all three splitting criterion in this case gave same value. This might be due to the same type of information gained from all the impurity checks. However this is not always the case as from weather dataset we can see that they give different accuracy values too.

VI. CONCLUSION

From the results obtained from various operations on the two datasets, it can be concluded that the decision tree tend to overfit with no stopping criterion. Also the overfitted dataset tend to perform bad eventhough it tries to capture most of infromation from the dataset. Similarly, the overfitted tree is very deep in nature which makes it computationally heavy to make decision as a lot of nodes need to be treaversed to reach a leaf node that can actually make some decision. Using Stopping Criterion for

early stopping can lead to shallower trees with no overfitting can increase efficiency of the model. It also shows that neither too deep nor too shallow tree can give better accuracy because they fall in underfitting and overfitting cases respectively. The appropriate max-depth of the tree can also differ based on the test and train set selected, their selection ratio and so on. Also the decision tree classifier can give a slightly different accuracy values even the training and testing occurs on same test and train data. Also the cross validation sets can be used to find the best accuracy and parameters for the tree in the particular dataset.

To further increase the accuracy and effectiveness of the tree, post pruning can be done like the cost-complexity pruning which will reduce the tree depth in more effective way than the stopping criterion used in this report.

APPENDIX A FLOW CHART

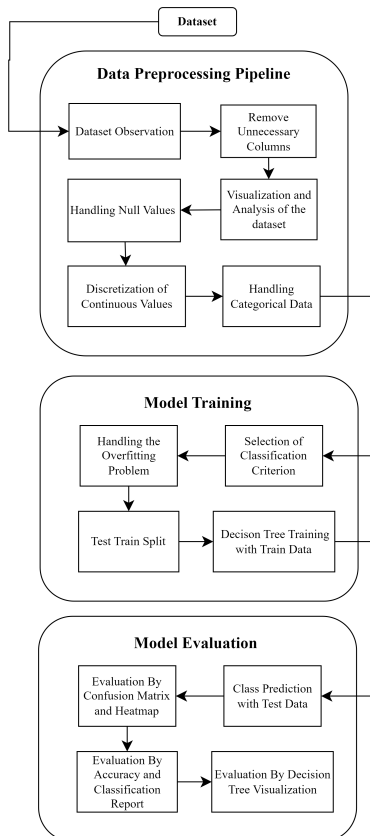


FIGURE 1. System Block Diagram

APPENDIX B THEORY DIAGRAMS

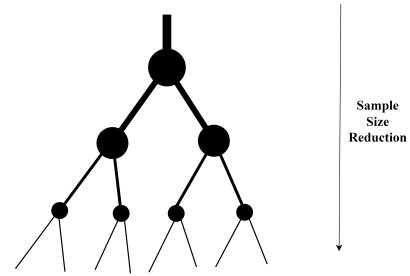


FIGURE 2. Decreasing Thickness of Decision Tree at Each Depth

APPENDIX C WEATHER DATASET FIGURES

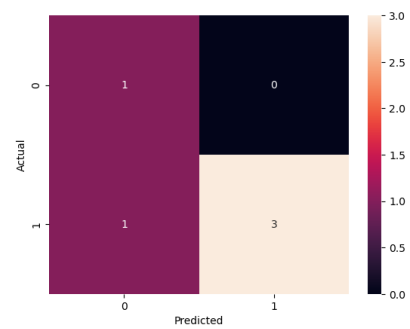


FIGURE 3. Heatmap of Weather Dataset with Entropy as Splitting Criterion and No max-depth provided

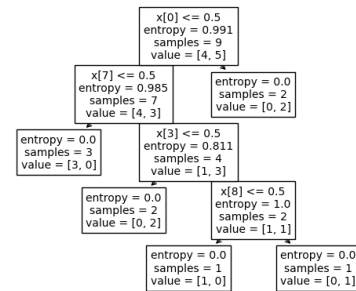


FIGURE 4. Tree Diagram of Weather Dataset with Entropy as Splitting Criterion and No max-depth provided

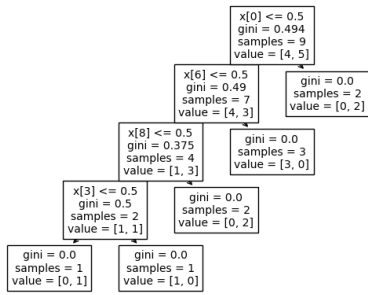


FIGURE 5. Tree Diagram of Weather Dataset with Gini as Splitting Criterion and No max-depth provided

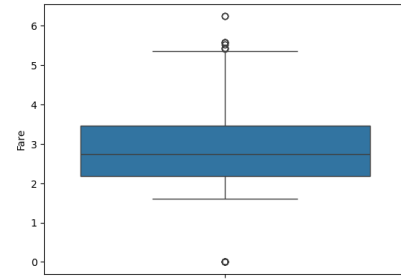


FIGURE 9. Box Plot of Fare Feature Before Handling Outliers

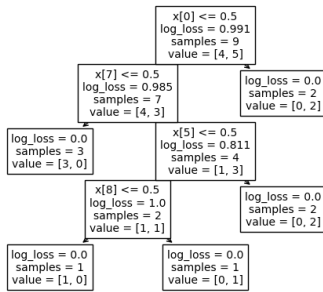


FIGURE 6. Tree Diagram of Weather Dataset with Log-Loss as Splitting Criterion and No max-depth provided

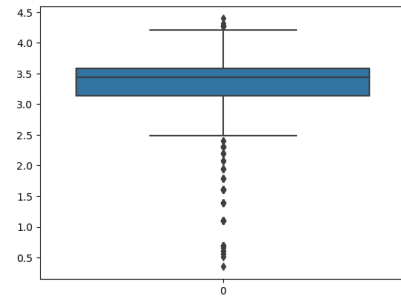


FIGURE 10. Box Plot of Age Feature After Handling Outliers

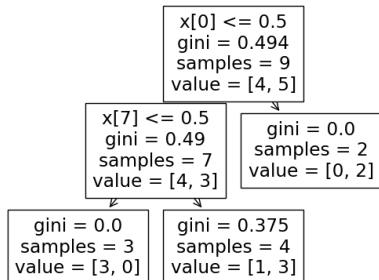


FIGURE 7. Tree Diagram of Weather Dataset with best parameters combination from Grid CV analysis

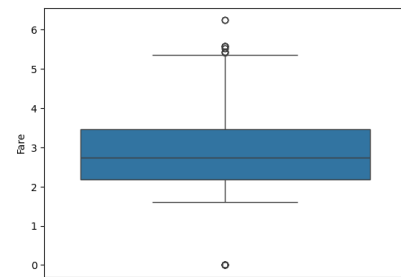


FIGURE 11. Box Plot of Fare Feature After Handling Outliers

APPENDIX D TITANIC DATASET FIGURES

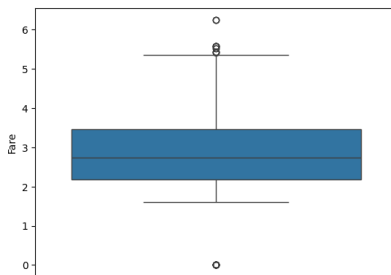


FIGURE 8. Box Plot of Age Feature Before Handling Outliers

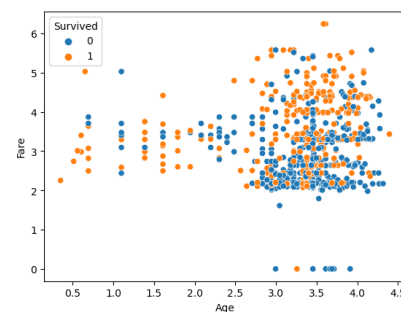


FIGURE 12. Scatter plot of Fare vs Age

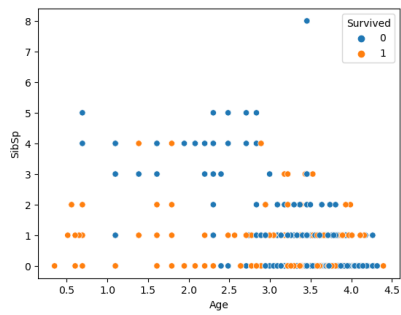


FIGURE 13. Scatter plot of SibSp vs Age

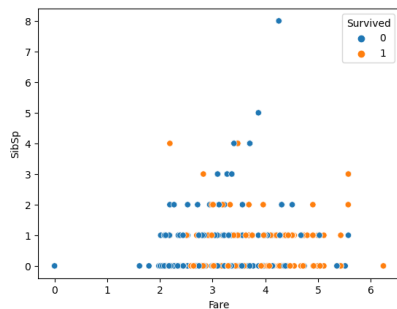


FIGURE 14. Scatter plot of SibSp vs Fare

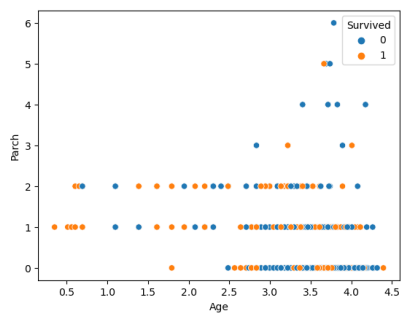


FIGURE 15. Scatter plot of Parch vs Age

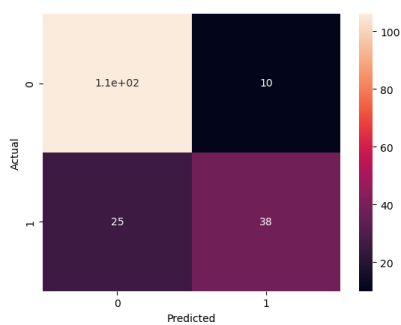


FIGURE 16. Heatmap of Titanic Dataset with Entropy as Splitting Criterion and No max-depth provided

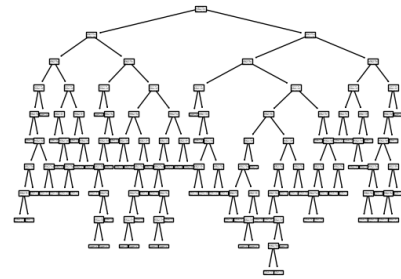


FIGURE 17. Tree Diagram of Titanic Dataset with Entropy as Splitting Criterion and no max-depth provided

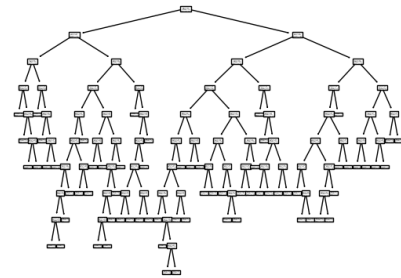


FIGURE 18. Tree Diagram of Titanic Dataset with Gini as Splitting Criterion and no max-depth provided

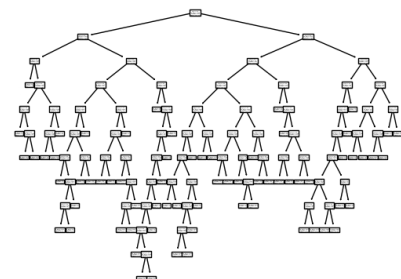


FIGURE 19. Tree Diagram of Titanic Dataset with Log-loss as Splitting Criterion and no max-depth provided

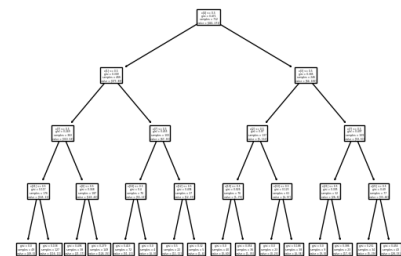


FIGURE 20. Tree Diagram of Titanic Dataset with best parameters combination from Grid CV analysis

APPENDIX E FORMULA EQUATIONS

$$Gini = 1 - \sum_0^c p_i^2 \quad (1)$$

where,
c = Total number of labels/classes,
 p_i = Probability of an item belonging to class i,

$$Entropy = - \sum_0^c p_i * \log_2(p_i) \quad (2)$$

where,
c = Total number of labels/classes,
 p_i = Probability of an item belonging to class i,

$$CE = 1 - \max[p_i(t)] \quad (3)$$

where,
i = proportion of instances belonging to class

$$LogLoss = -\frac{1}{N} \sum_i N \sum_j C_{y_{ij}} * \log(p_{ij}) \quad (4)$$

where,
N is the number of instances
C is the number of classes
 y_{ij} is 1 if instance i belongs to class j and 0 otherwise
 p_{ij} is the predicted probability that instance i belong to class j

$$NumberOfNodes = 2^{(depth+1)} - 1 \quad (5)$$

$$Accuracy = \frac{f_{11} + f_{00}}{f_{11} + f_{10} + f_{01} + f_{00}} \quad (6)$$

where,
 f_{11} = No. of True-Positive values,
 f_{00} = No. of True-Negative values,
 f_{10} = No. of False-Positive values,
 f_{01} = No. of False-Negative values,

$$Precision = \frac{f_{11}}{f_{11} + f_{00}} \quad (7)$$

where,
 f_{11} = No. of True-Positive values,
 f_{00} = No. of True-Negative values,

$$Recall = \frac{f_{11}}{f_{11} + f_{01}} \quad (8)$$

where,
 f_{11} = No. of True-Positive values,
 f_{01} = No. of False-Negative values,

$$F_1Score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (9)$$

APPENDIX F CODE SNIPPETS

```

1  import numpy as np
2  import pandas as pd
3  import math
4
5  def calc_entropy(series):
6      if series.empty:
7          return 0
8      counts = series.value_counts()
9      total = series.size
10     entropy = 0
11     for index in counts.index:
12         prob = counts[index] / total
13         entropy -= prob * math.log2(prob)
14
15     return entropy
16
17 def calc_collective_entropy(dataframe,
18                             split_pos, feature_name, class_name):
19     df1 = dataframe.loc[dataframe[
20         feature_name] < split_pos]
21     df2 = dataframe.loc[dataframe[
22         feature_name] >= split_pos]
23     df1_entropy = calc_entropy(df1[
24         class_name])
25     df2_entropy = calc_entropy(df2[
26         class_name])
27     df1_total = df1[class_name].size
28     df2_total = df2[class_name].size
29     grand_total = df1_total + df2_total
30     return (df1_total / grand_total) *
31     df1_entropy + (df2_total / grand_total) *
32     df2_entropy
33
34 def best_split_position(dataframe):
35     feature_name = dataframe.columns[0]
36     class_name = dataframe.columns[1]
37     unique_array = np.sort(dataframe[
38         feature_name].unique())

```

```

30     entropy_values = [
        calc_collective_entropy(dataframe, value,
        feature_name, class_name) for value in
        unique_array]
31     index = entropy_values.index(np.min(
        entropy_values))
32     print(entropy_values[index])
33     print(unique_array[index])
34     return unique_array[index]

```

Listing 1. Code Snippet For Best Split positon on Continuous Data

```

1     parameter = {
2         'criterion' : ['gini', 'entropy', '
        log_loss'],
3         'splitter' : ['best', 'random'],
4         'max_depth' : [None, 1,2,3,4,5,6,7,8,9],
5         'min_samples_split' : [2, 5, 10],
6         'max_features' : ['sqrt', 'log2']
7     }

```

Listing 2. Grid Search CV parameters

APPENDIX G TABLES

TABLE 1. Classification Report of Weather Dataset with Entropy as Splitting Criterion and No max-depth provided

	Precision	Recall	F1-Score	Support
No	0.50	1.00	0.67	1
Yes	1.00	0.75	0.86	4
Accuracy			0.80	5
Macro avg	0.75	0.88	0.76	5
Weighted avg	0.90	0.80	0.82	5

TABLE 2. Accuracy obtained using different Splitting Criterion for different max-depths for Weather Dataset

Max Depth	Splitting Criterion		
	Entropy	Gini	Log-loss
1	0.6	0.6	0.6
2	0.6	0.8	0.8
3	0.8	0.6	0.8
4	0.6	0.8	0.8
5	0.8	0.8	0.8
6	0.6	0.6	0.8
7	0.8	0.8	0.8
8	0.8	0.8	0.8
9	0.8	0.8	0.8
10	0.8	0.8	0.8

TABLE 3. Predicted Probabilities obtained using max-depth as 1 and critetion as entropy for Weather Dataset

0.57142857	0.42857143
0.57142857	0.42857143
0.00000000	1.00000000
0.00000000	1.00000000
0.57142857	0.42857143

TABLE 4. Predicted Probabilities obtained using max-depth as 2 and critetion as entropy for Weather Dataset

1.00	0.00
1.00	0.00
0.00	1.00
0.00	1.00
0.25	0.75

TABLE 5. Predicted Probabilities obtained using max-depth as 3 and critetion as entropy for Weather Dataset

1.0	0.0
1.0	0.0
0.0	1.0
0.0	1.0
0.5	0.5

TABLE 6. Predicted Probabilities obtained using max-depth as 4 and critetion as entropy for Weather Dataset

1.0	0.0
1.0	0.0
0.0	1.0
0.0	1.0
0.0	1.0

TABLE 7. Missing Values in the Titanic Dataset Before Preprocessing

Column	Missing Values Count
Survived	0
Pclass	0
Sex	0
Age	177
SibSp	0
Parch	0
Fare	0
Embarked	2



TABLE 11. Gini and no.of sample of last 15 nodes when Gini is splitting criterion and no max-depth is provided in Titanic Dataset

Gini	No. of Samples
0.424	59
0.427	42
0.415	17
0.0	1
0.494	27
0.5	22
0.444	3
0.5	2
0.0	1
0.499	19
0.494	9
0.48	10
0.32	5
0.375	4
0.0	1

TABLE 8. Missing Values in the Titanic Dataset After Preprocessing

Column	Missing Values Count
Survived	0
Pclass	0
Sex	0
Age	0
SibSp	0
Parch	0
Fare	0
Embarked	0

TABLE 9. Classification Report of Titanic Dataset with Entropy as Splitting Criterion and No max-depth provided

	Precision	Recall	F1-Score	Support
0	0.81	0.91	0.86	116
1	0.79	0.60	0.68	63
Accuracy			0.80	179
Macro Avg	0.80	0.76	0.77	179
Weighted Avg	0.80	0.80	0.80	179

TABLE 12. Log-loss and no.of sample of last 15 nodes when log-loss is splitting criterion and no max-depth is provided in Titanic Dataset

Log Loss	No. of Samples
0.0	1
0.0	1
0.0	1
0.0	4
0.991	27
1.0	22
0.918	3
1.0	2
0.991	9
0.971	10
0.722	5
0.0	1
0.811	4

TABLE 10. Entropy and no.of sample of last 15 nodes when entropy is splitting criterion and no max-depth is provided in Titanic Dataset

Entropy	No. of Samples
0.722	5
1.0	2
0.0	1
0.0	1
1.0	1
0.991	27
1.0	22
0.918	3
1.0	2
0.998	19
0.991	9
0.971	10
0.722	5
0.0	1
0.811	4

TABLE 13. Accuracy obtained using different Splitting Criterion for different max-depths for Titanic Dataset

Max-Depth	Splitting Criterion		
	Gini	Entropy	LogLoss
1	0.7989	0.7989	0.7989
2	0.7374	0.7374	0.7374
3	0.8101	0.8101	0.8101
4	0.8101	0.8101	0.8101
5	0.8101	0.8101	0.8101
6	0.8101	0.8101	0.8101
7	0.7877	0.7877	0.7877
8	0.7989	0.7989	0.7989
9	0.7989	0.7989	0.7989
10	0.7989	0.7989	0.7989

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Professor Dinesh Kshetri Baniya (DBK sir) for his invaluable guidance and support throughout this lab exercise on Decision Tree Classification at IOE Thapathali Campus. His insightful feedback and encouragement have greatly enriched our understanding of the subject matter and enhanced the quality of this report. DBK sir has inspired us to undertake deep analysis of the topic, which has helped us gain valuable insights on every topic we explore.

We are also thankful to our classmates and peers for their collaboration and constructive discussions during the lab sessions. Their diverse perspectives and inputs have contributed significantly to the depth and breadth of our learning experience.

Additionally, we would like to acknowledge Department of Electronics and Computer Engineering, IOE, Thapathali Campus for providing the necessary resources and facilities that enabled us to conduct this research effectively.

Last but not least, we extend our appreciation to our families and friends for their unwavering encouragement and understanding during this academic endeavor.

REFERENCES

- [1] X. Zhao and X. Nie, "Splitting Choice and Computational Complexity Analysis of Decision Trees," *Entropy*, vol. 23, no. 10, p. 1241, Sep. 2021, doi: 10.3390/e23101241.
- [2] Y. Wang, C. Song and S. Xia, "Unifying the Split Criteria of Decision Trees Using Tsallis Entropy," *arXiv preprint arXiv:1511.08136*, 2016.
- [3] P. Berka and I. Bruha, "Discretization and grouping: pre-processing steps for Data Mining," in *Principles of Data Mining and Knowledge Discovery*, Lecture Notes in Computer Science, vol. 1704, Berlin, Heidelberg: Springer, 2000, pp. 375-382. doi: 10.1007/BFb0094825.
- [4] P. N. Tan, M. Steinbach, A. Karpatne, and V. Kumar, "Introduction to Data Mining," India: Pearson, 2009.



RIJAN POKHREL is currently pursuing a Bachelor's Degree in Electronics and Information Engineering at IOE Thapathali Campus, as of 2024. His passion lies in leveraging AI to revolutionize industries and solve complex problems. He is a motivated learner, constantly staying updated with the latest advancements in AI. Currently exploring quantum machine learning and proficient in app development using Flutter and backend development with Firebase.

Passionate about mathematics, statistics, and machine learning, R. Pokhrel possesses a keen interest in discovering hidden insights from data. He aspires to bridge the financial sector with automated technology through artificial intelligence. Additionally, he is a strong advocate for free speech and is deeply interested in politics and the stock market.



SABIN ACHARYA is an undergraduate student studying Electronics, Communication and Information Engineering in Institute of Engineering, Thapathali Campus, Tribhuvan University graduating in April 2025.

He is a Tech-enthusiast with keen interests in circuits and electronics along with Machine Learning and Software Development. He has skills like Android Application Development with Flutter, Backend with Node.js and Machine Learning Skills with Python. His recent projects include "Virtual Simulation of Hand Gestures Using Hall Effect Sensors". Apart from academics, he is keenly interested in portrait sketching and music.

Mr. Acharya was the recipient of Semester Topper Award from Thapathali Campus in the respective fields for 4 times till March 2024.

...